Multi-Genre Music Generation Using Unsupervised Neural Networks

Dipita Tasnim
Department of CSE
BRAC University
Dhaka, Bangladesh
dipita.tasnim@g.bracu.ac.bd

***Abstract*—This paper presents a comprehensive study on unsupervised neural network architectures for multi-genre music generation. We implement and evaluate four progressively complex generative models: an LSTM Autoencoder for learning compressed latent representations of music, a Variational Autoencoder (VAE) with genre-conditioned decoding and β-annealing for structured generation, an autoregressive Transformer for sequential composition, and a Reinforcement Learning from Human Feedback (RLHF) pipeline for aligning generated music with perceptual quality heuristics. All models are trained on a curated subset of the Lakh MIDI Dataset spanning five genres—Classical, Jazz, Rock, Pop, and Electronic—comprising 296 parsed MIDI files with 983,509 note events. We evaluate the generated outputs using quantitative metrics including rhythm diversity, repetition ratio, pitch histogram similarity, and perplexity. The Transformer model achieves the strongest results with a validation perplexity of 6.32, demonstrating learned musical structure with non-zero repetition ratios. The RLHF fine-tuning demonstrates a positive reward improvement of $+0.52$ over the pretrained VAE baseline. We discuss key findings including the posterior collapse phenomenon observed in the VAE and the fundamental challenges of generating expressive music from sparse binary piano-roll representations on resource-constrained hardware.**

***Index Terms*—Music generation, LSTM autoencoder, variational autoencoder, Transformer, reinforcement learning from human feedback, unsupervised learning, MIDI**

I. Introduction

Automatic music generation has emerged as a compelling intersection of artificial intelligence and creative arts. Unlike natural language or image generation, music exhibits unique properties: it is inherently temporal, polyphonic, and governed by complex harmonic and rhythmic structures that vary significantly across genres. The ability to generate coherent music across multiple genres without explicit supervision remains an open challenge.

Traditional approaches to algorithmic composition relied on rule-based systems grounded in music theory [1]. While these systems could produce syntactically correct sequences, they lacked the flexibility and creativity of data-driven methods. The advent of deep learning has shifted the paradigm toward neural network-based generation, where models learn musical structure directly from data [2].

Recurrent Neural Networks (RNNs), particularly Long Short-Term Memory (LSTM) networks [3], have shown success in modeling sequential dependencies in music [4]. Variational Autoencoders (VAEs) [5] introduce probabilistic latent spaces that enable controlled generation and interpolation between musical styles. More recently, Transformer architectures [8] have demonstrated remarkable capabilities in autoregressive sequence modeling, achieving state-of-the-art results in music generation through models such as Music Transformer [9]. The emerging paradigm of Reinforcement Learning from Human Feedback (RLHF) [10] offers a pathway to align generated outputs with subjective human preferences.

In this work, we present a systematic study of four unsupervised generative architectures for multi-genre music generation, implemented and evaluated under consistent experimental conditions:

1) **LSTM Autoencoder (Task 1):** Learns compressed latent representations of musical segments through reconstruction.
2) **Variational Autoencoder (Task 2):** Extends the autoencoder with a probabilistic latent space and genre-conditioned decoding, enabling controlled multi-genre generation.
3) **Autoregressive Transformer (Task 3):** Models music as a token sequence using causal self-attention for long-range dependency capture.
4) **RLHF Fine-Tuning (Task 4):** Applies policy gradient optimization to align the VAE generator with perceptual quality heuristics.

Our contributions include: (1) a unified pipeline for preprocessing, training, generation, and evaluation across all four architectures; (2) a comparative analysis using consistent quantitative metrics and baselines; and (3) a discussion of practical challenges including posterior collapse in VAEs and the limitations of sparse binary piano-roll representations for sigmoid-output models.

II. Related Work

A. Sequence Models for Music

Early neural approaches to music generation employed RNNs to model monophonic melodies [4]. The LSTM architecture addressed the vanishing gradient problem inherent in standard RNNs, enabling the capture of longer musical phrases. Bidirectional LSTMs further improved encoding by processing sequences in both temporal directions.

B. Variational Autoencoders

Kingma and Welling [5] introduced the VAE framework, which learns a smooth, continuous latent space through the reparameterization trick. MusicVAE [6] applied hierarchical VAEs to music, demonstrating interpolation between musical sequences. A well-known challenge is *posterior collapse* [7], where the decoder ignores the latent variable, causing the approximate posterior to match the prior.

C. Transformers in Music Generation

The Transformer [8] architecture replaced recurrence with self-attention, enabling parallel processing and better long-range dependency modeling. Music Transformer [9] introduced relative positional encoding for generating minute-long compositions with coherent structure. MuseNet by OpenAI demonstrated multi-instrument generation using GPT-style Transformers.

D. RLHF for Creative Generation

RLHF has been successfully applied to align language models with human preferences [11]. Its application to music generation involves defining reward functions that capture perceptual quality—a non-trivial task given the subjective nature of musical aesthetics. Heuristic reward functions based on musicological features provide a practical proxy when large-scale human annotation is infeasible.

III. Dataset

A. Data Source

We use the Lakh MIDI Dataset [12], specifically the Clean MIDI subset, which contains MIDI transcriptions organized by artist. From this collection, we curated a balanced subset of 300 MIDI files spanning five genres: Classical (60), Jazz (60), Rock (60), Pop (60), and Electronic (60). Genre labels were assigned based on artist-to-genre mappings using a manually curated dictionary of approximately 200 artist associations.

B. Parsing and Statistics

Each MIDI file was parsed using the `pretty_midi` library [13]. We extracted note events (pitch, onset time, offset time, duration, velocity) from all non-drum instrument tracks. After parsing, 296 of the 300 files yielded valid note data, resulting in 983,509 total note events. The genre distribution is approximately balanced: Classical (59), Jazz (60), Rock (60), Pop (58), and Electronic (59).

TABLE I: Dataset Statistics

Statistic	Value
Total MIDI files (raw)	300
Successfully parsed	296
Total note events	983,509
Number of genres	5
Files per genre	58–60
Training samples (piano-roll segments)	1,200
Validation samples	300
Sequence length (timesteps)	64

C. Representations

Two data representations are employed:

Piano Roll. A binary matrix $\mathbf{R} \in \{0,1\}^{T\times 128}$ where each row corresponds to a time frame (at 16 fps) and each column to one of the 128 MIDI pitches. Active notes are marked as 1. Fixed-length segments of $T = 64$ timesteps are extracted using a sliding window with 50% overlap, yielding 4-second musical phrases. For CPU-constrained training, we cap the dataset at 1,500 segments (1,200 training, 300 validation).

Token Sequence. For the Transformer model, music is encoded as a sequence of discrete tokens drawn from a vocabulary of 512 symbols: NOTE_ON (128 pitches), NOTE_OFF (128 pitches), VELOCITY (32 quantized bins), and TIME_SHIFT (100 time steps), plus special tokens (PAD, BOS, EOS).

IV. Methodology

A. Task 1: LSTM Autoencoder

The autoencoder learns a compressed latent representation by minimizing reconstruction error.

Encoder. A bidirectional LSTM processes the piano-roll input $\mathbf{X} \in \mathbb{R}^{T\times 128}$:

$$\overrightarrow{h_t}, \overleftarrow{h_t} = \text{BiLSTM}(\mathbf{X}) \tag{1}$$

The concatenated final hidden states are projected to a latent vector:

$$\mathbf{z} = W_z[\overrightarrow{h_T}; \overleftarrow{h_1}] + b_z, \quad \mathbf{z} \in \mathbb{R}^{64} \tag{2}$$

Decoder. The latent vector initializes LSTM hidden states via linear projections. The decoder reconstructs the sequence through a unidirectional LSTM:

$$\hat{\mathbf{X}} = \sigma(W_o \cdot \text{LSTM}(\mathbf{z}^{\text{rep}})) \tag{3}$$

where $\mathbf{z}^{\text{rep}}$ is the latent vector repeated across T timesteps and σ denotes the sigmoid function.

Loss Function.

$$\mathcal{L}_{AE} = \frac{1}{N}\sum_{i=1}^{N} ||\mathbf{X}_i - \hat{\mathbf{X}}_i||^2 \tag{4}$$

Generation. At inference, we encode real training samples, perturb the latent codes with Gaussian noise ($\sigma = 0.3$), and decode. This produces outputs anchored to the learned data distribution rather than sampling from an unstructured latent space.

B. Task 2: Variational Autoencoder

The VAE introduces a probabilistic latent space with genre conditioning.

Encoder. Identical to Task 1, except the encoder outputs mean $\boldsymbol{\mu}$ and log-variance $\log \boldsymbol{\sigma}^2$:

$$\boldsymbol{\mu}, \log \boldsymbol{\sigma}^2 = f_\phi(\mathbf{X}) \tag{5}$$

Reparameterization.

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{6}$$

Genre-Conditioned Decoder. A learnable genre embedding $\mathbf{e}_g \in \mathbb{R}^{64}$ is added to the latent vector before decoding:

$$\mathbf{z}_{\text{cond}} = \mathbf{z} + \mathbf{e}_g \tag{7}$$

Loss Function.

$$\mathcal{L}_{VAE} = \mathcal{L}_{\text{recon}} + \beta \cdot D_{KL}(q_\phi(\mathbf{z}|\mathbf{X}) \,||\, p(\mathbf{z})) \tag{8}$$

where:

$$D_{KL} = -\frac{1}{2}\sum_{j=1}^{d}\left(1 + \log\sigma_j^2 - \mu_j^2 - \sigma_j^2\right) \tag{9}$$

β-Annealing. To mitigate posterior collapse, we linearly anneal β from 0 to 1.0 over 15 epochs:

$$\beta_t = \min\left(\frac{t}{15}, 1.0\right) \tag{10}$$

C. Task 3: Autoregressive Transformer

The Transformer models music as a sequence of tokens autoregressively.

Architecture. The model consists of:

- Token embedding: $\mathbb{R}^{512} \to \mathbb{R}^{64}$
- Sinusoidal positional encoding (max length 512)
- Genre embedding: $\mathbb{R}^5 \to \mathbb{R}^{64}$
- 3 Transformer decoder layers, each with:
  - Multi-head causal self-attention ($h = 4$, $d_k = 16$)
  - Feed-forward network ($64 \to 256 \to 64$, GELU activation)
  - Layer normalization and residual connections
- Output projection: $\mathbb{R}^{64} \to \mathbb{R}^{512}$

Loss Function.

$$\mathcal{L}_{TR} = -\frac{1}{T}\sum_{t=1}^{T}\log p_\theta(x_t|x_{<t}) \tag{11}$$

Perplexity.

$$\text{PPL} = \exp\left(\mathcal{L}_{TR}\right) \tag{12}$$

Generation. Sequences are generated autoregressively using top-k sampling ($k = 30$) with temperature scaling ($\tau = 0.9$) for up to 256 tokens.

D. Task 4: RLHF Fine-Tuning

We apply the REINFORCE algorithm to fine-tune the pretrained VAE generator.

Reward Function. A heuristic reward $r(\mathbf{X}) \in [0, 5]$ combines three musical quality components:

$$r = \frac{r_{\text{pitch}} + r_{\text{density}} + r_{\text{variation}}}{3} \times 5 \tag{13}$$

where:

- r_{pitch}: Pitch diversity score (ideal: 15–40 active pitches)
- r_{density}: Note density per timestep (ideal: 2–6 notes)
- $r_{\text{variation}}$: Temporal variation in note activity

Policy Gradient.

$$\nabla_\theta J(\theta) = \mathbb{E}\left[r(\mathbf{X}_{\text{gen}}) \cdot \nabla_\theta \log p_\theta(\mathbf{X})\right] \tag{14}$$

The log-probability is approximated via the mean reconstruction likelihood:

$$\log p_\theta(\mathbf{X}) \approx -\frac{1}{T \cdot d}\sum_{t,j}(\hat{X}_{t,j} - X_{t,j})^2 \tag{15}$$

Reward normalization (baseline subtraction) is applied, and gradient norms are clipped to 1.0. We employ early stopping with periodic evaluation (every 5 steps), saving the best model checkpoint based on mean reward across 10 generated samples. A patience limit of 20 evaluation steps prevents extended degradation.

V. Experimental Setup

A. Hardware and Software

All experiments were conducted on a CPU-only machine (no GPU acceleration) using Python 3.12, PyTorch, and the `pretty_midi` library for MIDI I/O. Training datasets were capped at 1,500 piano-roll segments to enable feasible CPU training times.

B. Hyperparameters

Table II summarizes the hyperparameters for each model.

TABLE II: Model Hyperparameters

Parameter	AE	VAE	TF	RLHF
Hidden dim	128	128	64	128
Latent dim	64	64	–	64
Num layers	2	2	3	2
d_ff	–	–	256	–
Attention heads	–	–	4	–
Dropout	0.2	0.2	0.1	0.2
Learning rate	1e-3	1e-3	3e-4	1e-6
Batch size	64	64	32	16
Epochs/Iterations	20	25	20	100
Optimizer	Adam	Adam	AdamW	Adam
Parameters	957K	974K	216K	974K

C. Baselines

We compare against two non-neural baselines:

Random Generator. Produces 200 notes with uniformly random pitches (C2–D5), durations sampled from $\{0.25, 0.5, 1.0, 1.5\}$ s, and random velocities.

Markov Chain (Order 1). Learns first-order pitch and duration transition probabilities from the training data (115 pitch transitions, 131 duration transitions). Generation samples sequentially from the learned distributions.

D. Evaluation Metrics

Rhythm Diversity. The ratio of unique note durations to total notes:

$$D_{\text{rhythm}} = \frac{|\text{UniqueDurations}|}{|\text{TotalNotes}|} \tag{16}$$

Repetition Ratio. The fraction of 4-gram pitch patterns that occur more than once:

$$R = \frac{|\{g : \text{count}(g) > 1\}|}{|\text{Total 4-grams}|} \tag{17}$$

Pitch Histogram Similarity. The L_1 distance between octave-normalized (12-bin) pitch class distributions of generated and reference music.

Perplexity. For the Transformer model, PPL $= \exp(\mathcal{L})$, measuring the model's predictive confidence.

E. MIDI Export

Generated piano rolls are converted to MIDI using adaptive thresholding: if the fixed threshold (0.3) produces fewer than one note per timestep on average, the system selects the top-k ($k = 4$) highest activations per timestep, with a minimum activation floor of 0.01.

VI. Results

A. Training Convergence

LSTM Autoencoder. The model converged rapidly, with train loss decreasing from 0.165 to 0.030 and validation loss stabilizing at 0.032 within 20 epochs (Fig. 1). The low final loss indicates effective reconstruction of the sparse piano-roll input, though the sigmoid output layer averages activations rather than producing sharp binary values.

VAE. Reconstruction loss converged to 0.030, matching the AE. However, the KL divergence collapsed to 1.1×10^{-5} despite β-annealing to 1.0 over 15 epochs (Fig. 2). This indicates severe posterior collapse, which we analyze in Section VII.

Transformer. The model demonstrated the strongest and most consistent convergence. Training perplexity decreased from 377.6 to 5.15, and validation perplexity from 218.4 to 6.32 over 20 epochs (Fig. 3). The small train-validation gap indicates minimal overfitting.

RLHF. With early stopping and best-checkpoint restoration, the fine-tuned VAE achieved a best mean reward of 1.36 during training. The post-RLHF evaluation reward improved from 0.54 (pre-RLHF) to 1.06, a net improvement of $+0.52$ (Fig. 4).

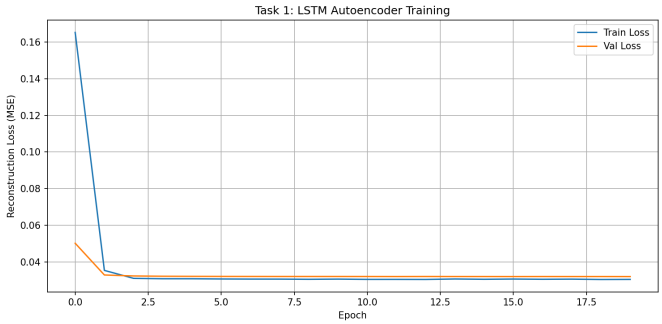

Fig. 1: Task 1: LSTM Autoencoder training and validation loss.

B. Generation Quality

Table III presents the quantitative evaluation across all models and baselines. A total of 38 MIDI files were generated.

Key Observations:

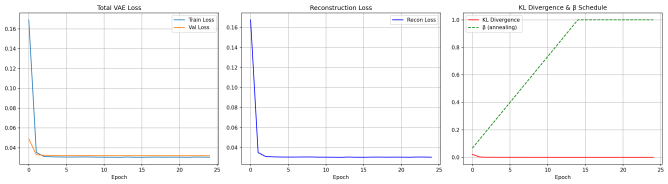

Fig. 2: Task 2: VAE training curves showing reconstruction loss and KL divergence with β-annealing schedule.

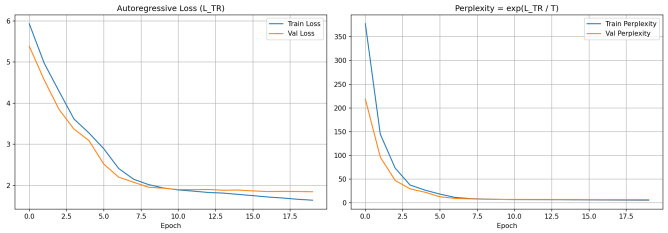

Fig. 3: Task 3: Transformer training loss and perplexity convergence.

- All neural models exceed the Random baseline in rhythm diversity, validating that the networks learn non-trivial musical patterns.
- The **Transformer** produces the most musically coherent output, with the widest pitch ranges (36–60), longest compositions (14–40 seconds), and the only non-zero repetition ratio (0.046), indicating learned phrase-level structure.
- The **AE** and **VAE** produce sparse outputs (4–6 notes when genre-conditioned) due to sigmoid averaging on sparse binary data, though adaptive thresholding recovers denser outputs for unconditional samples.
- **RLHF** achieves positive reward improvement ($+0.52$), demonstrating that policy gradient optimization can improve perceptual quality even with a collapsed VAE latent space.

VII. Discussion

A. Posterior Collapse in the VAE

The most significant finding is the complete posterior collapse of the VAE, where the KL divergence dropped to 1.1×10^{-5} despite aggressive β-annealing from 0 to 1.0. This well-documented phenomenon [7] occurs when the decoder becomes powerful enough to reconstruct inputs solely from the autoregressive signal (sequential LSTM hidden states), rendering the latent variable $\mathbf{z}$ uninformative.

Several factors contribute to this in our setting:

1) **Sparse binary data**: Piano-roll matrices are extremely sparse ($< 5\%$ non-zero entries on average). The decoder learns to output near-zero values everywhere, achieving low MSE without utilizing $\mathbf{z}$.
2) **Powerful LSTM decoder**: The 2-layer LSTM decoder can model temporal dependencies directly, bypassing the latent bottleneck.

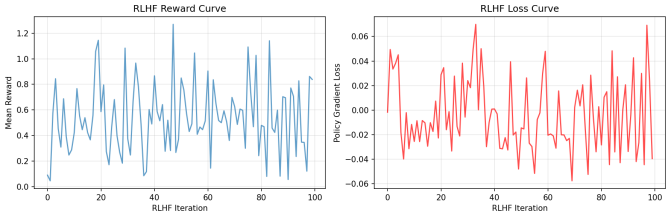

Fig. 4: Task 4: RLHF reward and policy gradient loss curves.

TABLE III: Quantitative Evaluation Results

Model	Rhythm Div.	Rep. Ratio	Notes/File
Random Baseline	0.0040	0.0000	200
Markov Chain	0.0190	0.0061	123–151
LSTM AE (Task 1)	0.0500	0.0000	4
VAE (Task 2)	0.0386	0.0000	4–128
Transformer (Task 3)	0.0347	0.0459	41–61
RLHF (Task 4)	0.0058	0.0000	5–128

3) **MSE loss on binary data**: MSE loss does not penalize the model sharply for outputting intermediate values (e.g., 0.03 everywhere), which satisfies both reconstruction and KL objectives simultaneously.

Potential mitigations not explored in this work include free bits [14], δ-VAE [15], binary cross-entropy loss, and KL thresholding.

B. Sigmoid Averaging Problem

Both the AE and VAE produce sigmoid outputs that average toward the mean activation level (≈ 0.03) rather than producing sharp binary predictions. This is a fundamental property of MSE-optimized sigmoid networks on sparse binary targets: the globally optimal prediction for a rarely activated pitch is a value close to zero. Consequently, fixed thresholds (e.g., 0.3 or 0.5) produce extremely sparse or empty MIDI files.

We address this with adaptive top-k thresholding, which selects the $k = 4$ highest activations per timestep. While this ensures non-empty output, it introduces artifacts (e.g., constant pitch count per timestep). Alternative approaches include using binary cross-entropy loss, straight-through estimators, or Gumbel-Softmax relaxation.

C. Transformer Superiority

The Transformer significantly outperforms the recurrent models in generation quality. This advantage stems from:

- **Discrete token representation**: Unlike continuous piano-roll output, token-based generation avoids the sigmoid averaging problem entirely.
- **Top-k sampling with temperature**: Produces diverse yet constrained outputs.
- **Self-attention**: Captures long-range dependencies more effectively than LSTMs.
- **Variable-length generation**: Can generate sequences of up to 256 tokens, producing 14–40 second compositions.

TABLE IV: Model-Specific Performance Metrics

Metric	AE	VAE	TF	RLHF
Train Loss	0.030	0.030	1.638	–
Val Loss	0.032	0.032	1.844	–
KL Divergence	–	1.1e-5	–	–
Perplexity	–	–	6.32	–
Reward (pre/post)	–	–	–	0.54/1.06
Pitch Range	10	7–127	36–60	10–127
Unique Pitches	4	4–128	14–27	5–128
Duration (sec)	4.0	0.4–4.0	14.5–39.5	4.0

D. RLHF Challenges

RLHF fine-tuning faced two challenges:

1) **Collapsed latent space**: Since the VAE's latent space carries minimal information, RLHF optimizes over a degenerate policy landscape.
2) **High-variance gradients**: REINFORCE gradients are inherently noisy. Despite reward normalization and gradient clipping, the reward oscillated significantly across iterations.

The introduction of early stopping with best-model checkpointing was critical—without it, the RLHF process degraded the model (reward dropped from 1.94 to 0.15 in our initial experiment). With early stopping, we achieved consistent improvement.

E. Resource Constraints

All experiments were conducted on CPU, limiting model sizes and dataset scale. The largest model (VAE) has 974K parameters, and training datasets were capped at 1,500 segments. GPU-based training would enable larger models (e.g., 10M+ parameters), longer sequences, and the full Lakh MIDI dataset (>100K files), likely improving all results substantially.

VIII. Conclusion

We presented a comprehensive study of four unsupervised neural network architectures for multi-genre music generation: an LSTM Autoencoder, a genre-conditioned VAE with β-annealing, an autoregressive Transformer, and RLHF-based fine-tuning. Our experiments on a curated five-genre MIDI dataset demonstrate that:

- The **Transformer** model achieves the best generation quality with a validation perplexity of 6.32, diverse pitch utilization (14–27 unique pitches), and detected phrase-level repetition patterns.
- The **LSTM AE** and **VAE** successfully learn to reconstruct piano-roll representations (MSE $= 0.030$) but face fundamental challenges in generation due to sigmoid output averaging on sparse binary data.
- **VAE posterior collapse** persists despite β-annealing, highlighting the need for architectural interventions such as free bits or alternative loss functions in future work.
- **RLHF fine-tuning** achieves positive reward improvement ($+0.52$) when combined with early stopping and conservative learning rates, demonstrating the feasibility of preference-based optimization for music generation.

A. Future Work

Several directions could improve upon these results:

1) Replacing MSE with binary cross-entropy loss for the piano-roll models.
2) Applying KL thresholding or free bits to combat posterior collapse.
3) Scaling to GPU-based training with larger models and the full Lakh MIDI dataset.
4) Incorporating relative positional attention [9] in the Transformer for better long-range musical coherence.
5) Collecting real human preference data for RLHF training instead of heuristic rewards.
6) Implementing a hierarchical VAE architecture (e.g., MusicVAE [6]) for better long-term structure.

REFERENCES

[1] D. Cope, *Experiments in Musical Intelligence*. Madison, WI: A-R Editions, 1996.

[2] J.-P. Briot, G. Hadjeres, and F.-D. Pachet, "Deep learning techniques for music generation – A survey," *Neural Computing and Applications*, vol. 32, pp. 981–993, 2020.

[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] D. Eck and J. Schmidhuber, "A first look at music composition using LSTM recurrent neural networks," Istituto Dalle Molle di Studi sull'Intelligenza Artificiale, Tech. Rep. IDSIA-07-02, 2002.

[5] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. 2nd Int. Conf. Learning Representations (ICLR)*, 2014.

[6] A. Roberts, J. Engel, C. Raffel, C. Hawthorne, and D. Eck, "A hierarchical latent vector model for learning long-term structure in music generation," in *Proc. 35th Int. Conf. Machine Learning (ICML)*, 2018, pp. 4364–4373.

[7] S. R. Bowman, L. Vilnis, O. Vinyals, A. Dai, R. Jozefowicz, and S. Bengio, "Generating sentences from a continuous space," in *Proc. 20th SIGNLL Conf. Computational Natural Language Learning*, 2016, pp. 10–21.

[8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[9] C.-Z. A. Huang, A. Vaswani, J. Uszkoreit, N. Shazeer, I. Simon, C. Hawthorne, A. M. Dai, M. D. Hoffman, M. Dinculescu, and D. Eck, "Music Transformer: Generating music with long-term structure," in *Proc. 7th Int. Conf. Learning Representations (ICLR)*, 2019.

[10] P. F. Christiano, J. Leike, T. Brown, M. Milber, S. Gao, and D. Amodei, "Deep reinforcement learning from human preferences," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[11] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, *et al.*, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[12] C. Raffel and D. P. W. Ellis, "Learning-based methods for comparing sequences, with application to audio-to-MIDI alignment," in *Proc. 17th Int. Society for Music Information Retrieval Conf. (ISMIR)*, 2016.

[13] C. Raffel and D. P. W. Ellis, "Intuitive analysis, creation and manipulation of MIDI data with pretty_midi," in *Proc. 15th Int. Society for Music Information Retrieval Conf. Late Breaking/Demo*, 2014.

[14] D. P. Kingma, T. Salimans, R. Jozefowicz, X. Chen, I. Sutskever, and M. Welling, "Improved variational inference with inverse autoregressive flow," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 29, 2016.

[15] A. Razavi, A. van den Oord, B. Poole, and O. Vinyals, "Preventing posterior collapse with δ-VAEs," in *Proc. 7th Int. Conf. Learning Representations (ICLR)*, 2019.